11 — B2B Bayi Portalı (Paspas Mimarisine Uyarlanmış)

> **Bağlam (v1.1):** Bu doküman **Faz 6 — B2B Bayi Portalı**'nın detaylı tasarımıdır. 10 alt modül, DB şeması, mimari karar (Postgres-tabanlı pasted örnek vs Paspas MySQL stack uyumlandırılması). Üst düzey faz haritası: `proje-teklifi/02-cozum-genel-bakis.md`. Özellik listesi: `proje-teklifi/05-fonksiyonel-kapsam.md` (76 özellik, MVP/Genişleme).

Strateji değişimi — neden bu doküman var

Daha önceki 02-musteri-kesif.md "yeni müşteri keşfi"ni öne aldı. Müşteri tabanı analizi sonrası strateji değişti:

> **Yeni müşteri kazanmak, mevcut müşteriyi elinde tutmaktan ve daha çok satmaktan 5-7 kat daha pahalı.**

Mevcut **35 aktif müşteri** (DB'de doğrulandı). İskonto dağılımı %0-45.60, ortalama %17.7 — yani:

İskonto	Tahmini segment	Davranış
%0-7	**Stratejik toptancılar/distribütörler**	Yüksek hacim, düşük marj, düzenli tekrar siparişi
%8-25	Orta segment otomotiv firmaları	Periyodik sipariş
%26-45	Tek seferlik / küçük otomotiv	Düşük tekrar

Bu 35 müşterinin %70'inde iletişim verisi eksik (görsel analizinden). **Önce mevcut müşteriyi düzene sok, sonra portal kur, sonra yeni müşteri a**

Doğru sıralama

```
[1] Mevcut müşteri enrichment (1-2 hafta)
       Eksik telefon/email/web doldur, segmentasyon çıkar
       ↓
[2] B2B Bayi Portalı v1 — read-only (3-4 hafta)
       Bayi login, katalog, sipariş geçmişi, cari hesap
       ↓
[3] B2B Bayi Portalı v2 — sipariş açma (2 hafta)
       Sepet + kredi limit + transaction'lı sipariş + audit
       ↓
[4] 5 pilot bayi onboarding + feedback (1 hafta)
       İstanbul Oto, Ankara Oto Market, Ege Oto, Güney, Karadeniz
       ↓
[5] Tam yayılım + yeni müşteri keşfi (Faz 3 ve sonrası)
```

Önemli mimari uyarı — yapıştırılan stratejideki hatalar

Yapıştırılan analiz metni doğru iş mantığına sahip ama **Paspas stack'ini bilmediği için yanlış teknolojiler önerdi**. Düzeltme:

Konu	Yapıştırılan öneri	Paspas gerçeği	Karar
DB	PostgreSQL	**MySQL 8**	MySQL kullan; kod örneklerini `mysql-core` 'a uyarla
ORM	Drizzle pg-core	**Drizzle mysql-core** (zaten kurulu)	mysql-core kullan
Backend framework	Next.js Server Actions	**Fastify 5 + Bun** (mevcut)	Mevcut Fastify'a yeni route'lar ekle
Auth	Auth.js v5 (NextAuth)	**Fastify JWT + httpOnly cookie** (mevcut)	Mevcut `auth/` modülünü genişlet — yeni `dealer` rolü
ID	int auto-increment	**char(36) UUID** (`randomUUID()`)	UUID kullan, mevcut convention
Tablo isimleri	`customers` , `products` , `orders`	**`musteriler` , `urunler` , `satis_siparisleri`** (Türkçe)	Türkçe konvansiyon
`iskonto` formatı	`0.05` (oran)	**`5.00`** (yüzde, decimal(5,2))	Yüzde — `final = lis` `* (1 - iskonto/100)`
Monorepo	Turborepo + pnpm workspaces	İki ayrı klasör (admin_panel, backend)	**Üçüncü klasör ekle:** `dealer_portal/`
Job queue	BullMQ + Redis	`setInterval` (P3 Madde 5'te kullanılan pattern)	Aynı pattern; gerek olursa BullMQ sonra
Email	Resend	Backend'de `mail/` modülü (Nodemailer/SMTP)	Mevcut modülü kullan, dış servis gerekirse Brevo
ERP "drafts" tablosu	`portal_order_drafts` → ERP sync	Direkt `satis_siparisleri/repository.ts:repoCreate`	**Drafts katmanı gereksiz** — Paspas backend zaten Fastify, portal direkt API çağırı
`vehicleCompat`	Tablo zaten varmış varsayılır	**Yok** — sıfırdan kurulması gerek	Yeni tablo: `urun_arac_uyumluluğ`

Paspas-uyumlu mimari

```
┌──────────────────────────────┐
│ paspas/dealer_portal/        │
│ (yeni Next.js 16 projesi)    │
│ Bayi UI: login, katalog,     │
│ sepet, sipariş, cari         │
└──────────────┬───────────────┘
               │
               │  HTTPS / REST + cookie auth
               ▼
┌──────────────────────────────┐
│ paspas/backend/              │
│ (mevcut Fastify 5)           │
│                              │
│ Yeni route prefix: /portal/  │
│ • POST /portal/auth/login    │
│ • GET  /portal/catalog       │
│ • POST /portal/cart          │
│ • POST /portal/orders        │
│ • GET  /portal/orders        │
│ • GET  /portal/cari          │
└──────────────┬───────────────┘
               │
               │  Drizzle ORM (mevcut)
               ▼
┌──────────────────────────────┐
│ MySQL (mevcut promats_erp)   │
│                              │
│ Mevcut: musteriler, urunler, │
│ satis_siparisleri,           │
│ siparis_kalemleri, hareketler│
│                              │
│ YENI:                        │
│ • portal_kullanicilari       │
│ • portal_oturumlari          │
│ • portal_sepet               │
│ • portal_sepet_kalemleri     │
│ • portal_audit               │
│ • urun_arac_uyumlulugu       │
│ • arac_modelleri             │
└──────────────────────────────┘
```

Tek doğruluk kaynağı: mevcut `promats_erp` MySQL'i. Portal kendi tabloları `portal_*` prefix'iyle ayrılır. Sipariş yazımı ortak — `repoCreate` (satis_siparisleri) reuse.

Yeni DB Şeması (Paspas konvansiyonu)

```
// backend/src/modules/portal/schema.ts (yeni modül)
import { sql } from 'drizzle-orm';
import { char, datetime, decimal, int, json, mysqlTable, text, tinyint, varchar } from 'drizzle-orm/mysql-core';

export const portalKullanicilari = mysqlTable('portal_kullanicilari', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  musteri_id: char('musteri_id', { length: 36 }).notNull(),
  email: varchar('email', { length: 255 }).notNull().unique(),
  password_hash: varchar('password_hash', { length: 255 }).notNull(),
  rol: varchar('rol', { length: 32 }).notNull().default('dealer'),
  is_active: tinyint('is_active', { unsigned: true }).notNull().default(1),
  must_change_pw: tinyint('must_change_pw', { unsigned: true }).notNull().default(1),
  last_login_at: datetime('last_login_at'),
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
  updated_at: datetime('updated_at').notNull().default(sql`CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`),
});

export const portalSepet = mysqlTable('portal_sepet', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  kullanici_id: char('kullanici_id', { length: 36 }).notNull(),
  musteri_id: char('musteri_id', { length: 36 }).notNull(),
  durum: varchar('durum', { length: 32 }).notNull().default('aktif'),
  // aktif | siparise_donustu | terk_edildi
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
  updated_at: datetime('updated_at').notNull().default(sql`CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`),
});

export const portalSepetKalemleri = mysqlTable('portal_sepet_kalemleri', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  sepet_id: char('sepet_id', { length: 36 }).notNull(),
  urun_id: char('urun_id', { length: 36 }).notNull(),
  arac_marka: varchar('arac_marka', { length: 64 }),
  arac_model: varchar('arac_model', { length: 128 }),
  arac_yili: int('arac_yili', { unsigned: true }),
  miktar: decimal('miktar', { precision: 12, scale: 4 }).notNull(),
  birim_fiyat: decimal('birim_fiyat', { precision: 12, scale: 2 }).notNull(),  // iskonto sonrası
  liste_fiyat: decimal('liste_fiyat', { precision: 12, scale: 2 }).notNull(),  // iskonto öncesi
  iskonto_yuzde: decimal('iskonto_yuzde', { precision: 5, scale: 2 }).notNull(),
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
});

export const portalAudit = mysqlTable('portal_audit', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  kullanici_id: char('kullanici_id', { length: 36 }),
  musteri_id: char('musteri_id', { length: 36 }),
  aksiyon: varchar('aksiyon', { length: 64 }).notNull(),
  // login_basarili, login_hatali, logout, sepete_ekle, siparis_olustur, fiyat_goster
  entity_tipi: varchar('entity_tipi', { length: 64 }),
  entity_id: varchar('entity_id', { length: 128 }),
  metadata: json('metadata'),
  ip_adresi: varchar('ip_adresi', { length: 45 }),
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
});

// Araç katalog — sıfırdan
export const aracModelleri = mysqlTable('arac_modelleri', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  marka: varchar('marka', { length: 64 }).notNull(),
  model: varchar('model', { length: 128 }).notNull(),
  govde_tipi: varchar('govde_tipi', { length: 32 }),  // sedan, hatchback, suv, station
  yil_baslangic: int('yil_baslangic', { unsigned: true }).notNull(),
  yil_bitis: int('yil_bitis', { unsigned: true }),
  notlar: varchar('notlar', { length: 500 }),
  is_active: tinyint('is_active', { unsigned: true }).notNull().default(1),
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
});

export const urunAracUyumlulugu = mysqlTable('urun_arac_uyumlulugu', {
  id: char('id', { length: 36 }).primaryKey().notNull(),
  urun_id: char('urun_id', { length: 36 }).notNull(),
  arac_model_id: char('arac_model_id', { length: 36 }).notNull(),
```

```
  notlar: varchar('notlar', { length: 500 }),
  created_at: datetime('created_at').notNull().default(sql`CURRENT_TIMESTAMP`),
});
```

Önemli: `iskonto` field'ı `musteriler` tablosunda **yüzde olarak saklanıyor** (5.00 = %5, 0.05 değil). Hesaplama:

```
const iskontoOrani = Number(musteri.iskonto) / 100; // 5.00 → 0.05
const finalFiyat = Number(urun.birim_fiyat) * (1 - iskontoOrani);
```

Sipariş akışı — drafts katmanı gereksiz

Yapıştırılan strateji "portal_order_drafts → ERP sync" katmanı önerdi. Paspas için **gereksiz**:

```
Müşteri portal'a sipariş yazar
  ↓
POST /portal/orders (Fastify route)
  ↓
Paspas backend transaction:
  1. Sepet kalemlerini doğrula
  2. Stok kontrol (stok_takip_aktif olanlar için)
  3. Kredi limit kontrol (musteriler.kredi_limit — yeni alan)
  4. satis_siparisleri/repository.ts:repoCreate çağır
  5. portal_audit'e yaz
  6. Sepet durumu → 'siparise_donustu'
  ↓
Sipariş gerçekten DB'de oluşur (SS-2026-XXXX)
  ↓
Geri rapor: müşteriye onay numarası + termin
```

`repoCreate` zaten transaction kullanıyor, kalemler oluşturuyor. Drafts katmanı eklemek lüzumsuz katman olur.

Auth — mevcut `auth/` modülünü genişlet

Yapıştırılan stratejideki Auth.js v5 gereksiz. Paspas zaten Fastify JWT + cookie kullanıyor (`access_token`):

```
// backend/src/modules/portal/auth.controller.ts (yeni)
import bcrypt from 'bcryptjs';
import type { RouteHandler } from 'fastify';
import { db } from '@/db/client';
import { portalKullanicilari, portalAudit } from './schema';
import { musteriler } from '@/modules/musteriler/schema';
import { eq, and } from 'drizzle-orm';

export const dealerLogin: RouteHandler<{ Body: { email: string; password: string } }> = async (req, reply) => {
  const { email, password } = req.body;
  const ip = req.ip;

  const [user] = await db
    .select()
    .from(portalKullanicilari)
    .where(and(eq(portalKullanicilari.email, email.toLowerCase()), eq(portalKullanicilari.is_active, 1)))
    .limit(1);

  if (!user) {
    await bcrypt.compare(password, '$2a$10$dummyhash'); // timing attack koruma
    return reply.code(401).send({ error: { message: 'invalid_credentials' } });
  }

  const ok = await bcrypt.compare(password, user.password_hash);
  if (!ok) {
    await db.insert(portalAudit).values({ id: randomUUID(), kullanici_id: user.id, aksiyon: 'login_hatali', ip_adresi: ip });
    return reply.code(401).send({ error: { message: 'invalid_credentials' } });
  }

  // Müşteri aktif mi?
  const [musteri] = await db.select().from(musteriler).where(eq(musteriler.id, user.musteri_id)).limit(1);
  if (!musteri || musteri.is_active !== 1) {
    return reply.code(403).send({ error: { message: 'musteri_pasif' } });
  }

  // Mevcut Fastify JWT plugin'ini kullan
  const token = req.server.jwt.sign({
    sub: user.id,
    musteri_id: user.musteri_id,
    role: 'dealer',
    email: user.email,
  }, { expiresIn: '7d' });

  reply.setCookie('access_token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    path: '/',
    maxAge: 7 * 24 * 60 * 60,
  });

  await db.insert(portalAudit).values({ id: randomUUID(), kullanici_id: user.id, musteri_id: user.musteri_id, aksiyon: 'login_basarili', ip_adresi: ip }
  await db.update(portalKullanicilari).set({ last_login_at: new Date() }).where(eq(portalKullanicilari.id, user.id));

  return reply.send({ ok: true, must_change_pw: user.must_change_pw === 1 });
};
```

`requireAuth` middleware'i zaten var (mevcut `authPlugin`). Yeni middleware: `requireDealerRole` — token'da `role === 'dealer'` kontrol ede müşterinin sadece kendi verisine erişimini sağlar.

Frontend — `dealer_portal/` yeni klasör

Mevcut `admin_panel/` ile **aynı seviyede** üçüncü bir Next.js 16 projesi:

```
paspas/
├── admin_panel/          (mevcut, yönetici UI)
├── backend/              (mevcut, Fastify)
├── dealer_portal/        (YENİ, bayi UI)
│   ├── package.json
│   ├── next.config.js
│   ├── src/
│   │   ├── app/
│   │   │   ├── login/page.tsx
│   │   │   ├── (dealer)/
│   │   │   │   ├── dashboard/page.tsx
│   │   │   │   ├── katalog/page.tsx
│   │   │   │   ├── sepet/page.tsx
│   │   │   │   ├── siparisler/page.tsx
│   │   │   │   └── cari/page.tsx
│   │   ├── integrations/
│   │   │   └── portal-api.ts (RTK Query — admin_panel pattern'iyle aynı)
│   │   └── components/
│   └── tailwind.config.js (admin_panel'den kopyala)
└── docs/
```

Tech stack (admin_panel ile uyumlu): - Next.js 16 + React 19 + Tailwind 4 - Redux Toolkit + RTK Query (admin_panel'deki gibi, Server Actions deği shadcn/UI (mevcut bileşenler kopyalanır) - Auth: `access_token` cookie + Fastify JWT - API çağrıları: `http://localhost:8078/portal/*`

4 haftalık sprint planı

Hafta 0 — Mevcut müşteri enrichment (1 hafta) ⚡

Önce 35 mevcut müşterinin verisini doldur:

Gün	İş
1-2	Google Places API setup + script: `scripts/enrich-musteriler.ts`
3	Pattern analizi (mevcut email formatları, telefon prefix'leri)
4	Hunter.io domain search ile eksik email'leri çek
5	Manuel doğrulama: yöneticiye 35 müşteri için "AI önerisi" panosu, tıkla onayla
6-7	Segment otomasyonu: %0-7 → toptancı, %8-25 → otomotiv, %26+ → küçük

Çıktı: 35 müşterinin %90+ verisi tamam, segmentleri belirli.

Hafta 1 — Portal backend + DB

Gün	İş
1	DB migration (`195_v1_portal_tablolari.sql`) — 7 yeni tablo
2	`backend/src/modules/portal/` modülü iskeleti (schema, repository, controlle router)
3	Auth: dealer login + JWT cookie + middleware (`requireDealerRole`)
4-5	Catalog endpoint: `/portal/catalog` filtre + iskonto uygulama (server-side)
6-7	Sepet endpoints: `POST /portal/cart/items` , `DELETE /portal/cart/items/:id`

Hafta 2 — Portal frontend + sipariş

Gün	İş
8-9	`dealer_portal/` Next.js 16 setup, login sayfası, dashboard skeleton
10-11	Katalog sayfası: marka/model/yıl filtreleri + ürün listesi (RTK Query)
12	Sepet sayfası: ekle/sil/güncelle
13-14	Checkout: kredi limit kontrol + `repoCreate` çağrısı + audit

Hafta 3 — Cari, sipariş geçmişi, polish

Gün	İş
15	Cari hesap sayfası: bakiye, vade, ekstre
16	Sipariş geçmişi + detay sayfası
17	Yeniden sipariş özelliği (en çok aldığı 5 ürün)
18	Email bildirim (sipariş alındı) — mevcut `mail/` modülü
19	Admin panel: `/admin/portal-kullanicilari` — yöneticinin bayi şifre yaratma
20-21	E2E test (Playwright — mevcut altyapı), güvenlik kontrolü, deploy

Hafta 4 — Pilot bayi onboarding

5 stratejik müşteri seçimi (DB'den iskonto %0-7 olanlar): - Yöneticinin elle seçim listesi - Her birine WhatsApp ile login bilgisi - 1 hafta feedback toplama - Düzeltmeler sonrası tüm bayilere açma

Ürün-araç uyumluluk verisi — kritik

Yeni tablo `arac_modelleri` boş başlar. Doldurma stratejisi:

Yöntem	Hız	Kalite
Manuel (yönetici)	Yavaş, 200-500 satır	Yüksek
Excel import (mevcut paspas üretimden)	Hızlı	Orta (validation gerek)
AI destekli (Claude/OpenAI) — "Toyota Corolla 2018 hangi paspas tipleri uyumlu"	Orta	Doğrulama gerek
Türkiye TUİK araç tescil verisi (public)	Yavaş ama tam	Yüksek

Pragmatik plan: İlk 30-40 araç modeli (en çok satan) manuel/Excel ile doldur — pazarın %80'i. Long-tail sonra.

Multi-channel sequence (Faz 2 sonrası)

Yapıştırılan stratejinin önerdiği toptancı sequence'i:

```
Gün 0:  Email + WhatsApp (paralel)
Gün 3:  WhatsApp follow-up + numune teklifi
Gün 7:  Telefon (manuel — satış temsilcisi)
Gün 14: Email — kampanya/fırsat
Gün 30: Aylık katalog/fiyat listesi
```

Bu Faz 4'e (yurt dışı + outreach) bağlanır. Şu an portal MVP odaklı.

Açık sorular

1. **`musteri.iskonto` formatı:** Şu an decimal(5,2) yüzde. UI'da 5.00 mu yoksa "%5" mi gösterilecek? Backend hesabı net.
2. **Kredi limit alanı:** `musteriler` tablosunda yok. Yeni alan ekleyelim mi (`kredi_limit decimal(12,2)` , `mevcut_bakiye decimal(12,2)`)?
3. **Pilot 5 bayi seçimi:** İskonto %0-7 olanlar otomatik mi seçilsin yoksa elle mi?
4. **Araç katalog veri kaynağı:** Excel export mevcut mu? Yoksa manuel + AI destekli mi başlanır?
5. **B2B portal subdomain:** `portal.paspas.com` mu, `bayi.paspas.com` mu? SSL setup?
6. **Şifre sıfırlama:** Email-based reset link, 24 saatlik token? Mail modülü hazır.
7. **Stok görünürlüğü:** Müşteri "5 adet kaldı" görsün mü, yoksa sadece "stokta var/yok"? B2B'de tam sayı yaygın.

Bağımlılıklar

- ✅ `musteriler` , `urunler` , `satis_siparisleri/repoCreate` mevcut
- ✅ Drizzle MySQL, Fastify JWT, Next.js 16 admin_panel pattern
- ✅ Playwright e2e altyapısı (P3 Madde 3)
- ⚠️ Mevcut müşteri enrichment scripti yok
- ⚠️ Araç katalog tablosu yok
- ⚠️ Kredi limit alanı `musteriler` 'de yok
- ⚠️ Bayi şifre yönetim UI'ı admin panelde yok

Toplam tahmini süre

Hafta	İçerik	Süre
0	Müşteri enrichment	1 hafta
1	Portal backend + auth + catalog	1 hafta
2	Frontend + sipariş akışı	1 hafta
3	Cari + geçmiş + admin tarafı + test	1 hafta
4	Pilot onboarding	1 hafta
Toplam	**MVP**	**5 hafta**

06-yol-haritasi.md'de bu, **Faz 1.5 (enrichment) + Faz 2 (portal)** olarak yer alır. Eski Faz 2 (talep AI) Faz 2.5'e kayar.

Bir sonraki adım

1. Bu doküman onaylanır mı?
2. Onaylanırsa: `06-yol-haritasi.md` güncellenir (B2B portal Faz 2 olarak kayar, lead discovery Faz 3'e iter)
3. Üstteki **7 açık soruya** cevap → Hafta 0 başlar